You said:

09_tic_tac_toe-mai...
File

Lab_4_VibeCoding...
PDF

give me the all tasks implementations prompt required in this task by task. just give me the prompts, the code part will be implemented by antigravity itself

Worked for 16s

ChatGPT said:

The assignment has **4 tasks**: fix the existing bugs, implement a persistent scoreboard, validate turns and moves, and add first-player selection with separate restart controls. Use the following prompts in Antigravity, one task at a time, against your existing Tic-Tac-Toe repository.

Task 1 — Fix win and draw detection

Writing

Inspect the existing Tic-Tac-Toe project and identify the exact causes of the win/draw detection bugs. Implement the following fixes without unnecessarily rewriting the project or changing its existing architecture.

1. Detect all 8 winning combinations: 3 rows, 3 columns, and both diagonals.
2. Check for a winner before checking whether the board is full. If the final move wins the game, announce the winner instead of declaring a draw.
3. Once a round ends in a win or draw, reject all further moves until the round is restarted.
4. Ensure the game state, displayed result, and move handling remain consistent after the round ends.

Inspect the existing game engine and rules modules, and make the changes in the appropriate files. Preserve the existing Pygame interface and random-move computer opponent.

Test all 8 winning lines, a full-board draw, a winning move that fills the last empty cell, and attempts

to play after a round has ended. Fix any failures you find. Do not implement the scoreboard or other new features yet.

Task 2 — Implement a persistent scoreboard

Writing

Implement a persistent scoreboard for the existing Tic-Tac-Toe game.

Requirements:

1. Track the total number of rounds won by X, rounds won by O, and draws.
2. Display the scoreboard visibly in the existing Pygame interface. Keep it readable and properly positioned without overlapping the board or other interface elements.
3. Update the appropriate score exactly once when a round ends. A round must never be counted more than once, even if the player continues clicking or the game processes additional events.
4. When restarting a round, clear the board and round-specific state while preserving all scoreboard totals.
5. Keep the scoreboard unchanged throughout a round until that round finishes.
6. The scoreboard must reset only when the player explicitly resets the entire match.

Modify the existing game engine and renderer as needed. Keep the implementation consistent with the current project structure and avoid duplicating state unnecessarily.

Test multiple consecutive rounds, wins by both X and O, draws, round restarts, and full-match resets. Confirm that scores persist correctly across round restarts and that each completed round increments exactly one category. Do not implement first-player selection or new restart controls yet.

Task 3 — Implement turn and move validation

Writing

Fix move validation in the existing Tic-Tac-Toe project.

Requirements:

1. When the player clicks an occupied cell, reject the move entirely.
2. An invalid click must not overwrite the existing symbol, change the current player, trigger the computer's move, or modify any game state.
3. Only accept moves that target an empty cell and are valid for the current turn.
4. Preserve the existing rule that the human player controls X and the computer controls O.
5. Ensure the computer never plays in an occupied cell and never makes a move after the round has ended.
6. Keep the win/draw detection and scoreboard behavior from the previous tasks working correctly.

Inspect the existing event handling, game engine, and AI modules to determine where validation belongs. Implement the fix in the appropriate location rather than adding superficial checks only to the user interface.

Test clicks on occupied cells for both symbols, repeated clicks on the same cell, valid moves, computer responses, and clicks after a round ends. Verify that rejected moves leave the board, turn, and scoreboard unchanged. Do not implement first-player selection or separate restart controls yet.

Task 4 — Add first-player choice and separate restart controls

Writing

Implement first-player selection and separate round-restart and full-match-reset controls in the existing Tic-Tac-Toe project.

Requirements:

1. Allow the player to choose whether X or O starts the round.
2. If X starts, allow the human player to make the first move.
3. If O starts, the computer must automatically make the first move as O. The human player must then play X.

4. Apply the selected starting symbol whenever a new round begins. Changing the selection must not corrupt an ongoing round or alter completed scoreboard totals.
5. Provide two clearly distinguishable controls:
   - Restart Round: clears the current board and resets round-specific state while preserving the scoreboard.
   - Reset Match: clears the current board, resets round-specific state, and sets all scoreboard totals to zero.
6. Ensure restarting a round or resetting a match correctly handles the selected starting player. If O starts, the computer should automatically make its opening move.
7. Preserve all existing functionality, including the eight winning combinations, correct win-before-draw evaluation, post-game move blocking, occupied-cell validation, and accurate scoreboard updates.
8. Integrate the controls into the existing Pygame interface. Keep the layout readable, avoid overlapping elements, and provide clear visual feedback for the selected starting player and completed rounds.

Inspect the existing main.py, game_engine.py, rules.py, ai.py, and renderer.py before implementing changes. Reuse the current architecture and avoid unnecessary rewrites or additional dependencies.

Test both starting-player choices, multiple consecutive rounds, both restart controls, wins and draws, occupied-cell clicks, and restarting immediately after a completed round. Verify that the scoreboard is preserved by Restart Round and cleared only by Reset Match. Fix any regressions discovered during testing.

Final verification prompt

Use this after all four tasks have been implemented.

Writing

Perform a complete integration review of the Tic-Tac-Toe project against every requirement in its README.md.

Verify that:
- All eight winning lines are detected.

- A win takes precedence over a draw when the final cell completes a winning line.
- No moves are accepted after a round ends.
- The scoreboard accurately records X wins, O wins, and draws exactly once per round.
- Round restarts preserve the scoreboard.
- Full-match resets clear the scoreboard.
- Occupied-cell clicks are rejected without changing the board or turn.
- X and O can each be selected to start, with the computer automatically playing first when O is selected.
- The selected starting player is applied correctly to subsequent rounds.
- Both restart controls work independently and correctly.
- The interface remains readable, with no overlapping controls or scoreboard elements.

Run the existing test suite, if present, and add or run focused tests for any unverified behavior. Fix failures and regressions. Do not remove working features, change the project's intended gameplay, or introduce unnecessary dependencies.

Finally, report which files were changed, which tests were executed, and any remaining issues. Do not claim a test passed unless it was actually executed successfully.

Execution order: Task 1 → Task 2 → Task 3 → Task 4 → Final verification.

Each prompt deliberately limits the scope of its task so Antigravity can implement and verify changes incrementally rather than rewriting the entire game in one attempt.